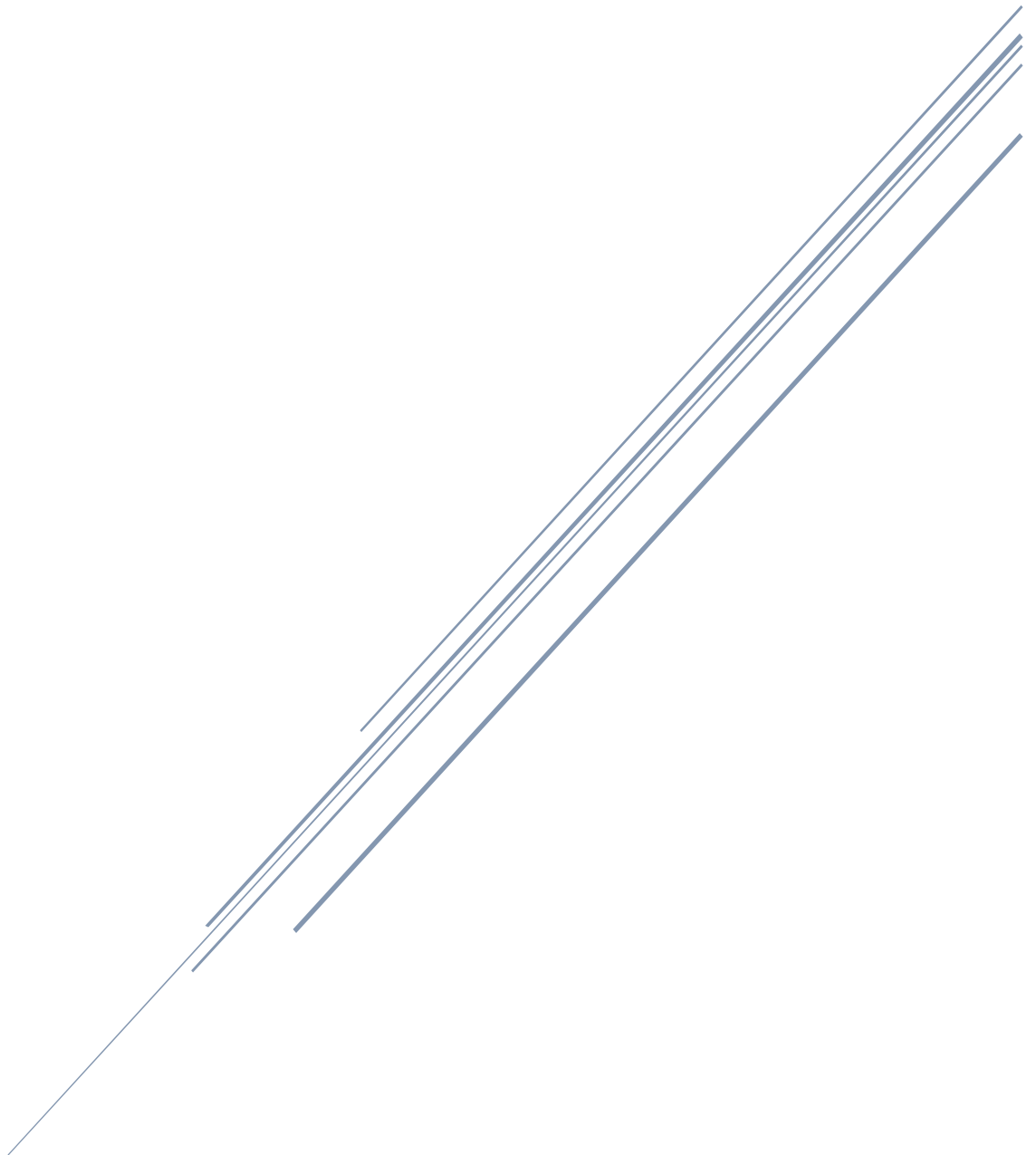


Design Document

By Beatrice Marro



Fontys University of Applied Science
S3 Software Engineer

Table of Contents

Contents

Table of Contents.....	1
Introduction to Software Architecture	2
Architecture Constraints and Design Decisions	2
Application Structure	3
Introduction to C4.....	4
Walking Skeleton	4
CI Setup	5
C1 (System Context).....	6
C2 (Containers and Technology Choices).....	7
C3 (Components and their Interactions)	8
C4 (Code Diagram)	9
Software Principles	10

Introduction to Software Architecture

The importance of having an architecture is vital to the planning and operations within an IT department to make a clear decision and communicate this efficiently among developers, to construct the system based on this choice. It is also fundamental to document this architecture for referral, to ensure that the correct architecture is implemented, by clearly defining it within the document, in a way that is easy and straight-forward to understand. It is also important to document any architectural choices so that also stakeholders other than developers have access to this information at any point of the project.

Architecture Constraints and Design Decisions

My architecture constraints consist of having to use a Rest API as a backend and having to create a frontend web application using a framework and implementing a design using CSS or a framework as well as using an SQL database.

Frontend:

For my frontend I am making a single page React JS Web Application. I am using Vite for improved and efficient development speeds. I am using Axios to access the data from my rest API. I will be using React Bootstrap as my frontend CSS framework for designing the user interface and to achieve a nice user experience through an intuitive and well-made design. I am using Node.js to setup a server-side environment to handle data processing, communication with APIs, authentication and improving the performance of my application. I am also using an external library called react-chartjs-2, used to display graphs from statistics figures in my web application.

Backend:

For my backend I will be using a Sprint Boot Java Rest API, for this I will be using HTTP as my communication protocol and JSON as the data response format. I am using Gradle for automating the creating of my application, and Lombok to reduce redundant code, and optimizing it by using annotations, such as the setting of getter and setter methods, constructors, and when applying inheritance. My architecture consists of 4 layers: Controllers (handle API requests and provide responses), Services (consist of logic and act as an intermediary between the controllers and repositories), Domain (consists of object classes) and Repositories (responsible for database CRUD interactions).

ORM:

As my object-relational mapping framework I am using JPA (Java Persistence API). In the case of aggregated queries, I am using custom JPQL queries.

Database:

For my database I will be using a relational Microsoft SQL Server database. I will be using encryption for the data in the database and salting and hashing for storing passwords securely using BCrypt. My database is stored in a local Docker container.

Unit Tests:

Unit tests are used in my Spring Boot Java Rest API to test the service classes of my application, using good and bad flow to ensure that the output of a method always returns the expected result. When a bad flow is used, typically this should return in the throwing of a custom-exception. For my business-layer unit tests I am using Mockito to mock the data from my repositories. I am using SonarQube for checking against technical debt, as to ensure that at least 80% of the service layer is being tested and that no bugs are present.

Application Structure

My application is structured in the following way:

1. **Repositories** are used to interact with the database using JPA.
2. **Base and Sub- classes**, these store the information of an object, there can vary from entities (used to interact with the database), to requests and responses (used as the input or output of the application).
3. **Services** are used as the interaction point between the repositories and the controller, these consist of CRUD actions, such as creating an article, any calculations and algorithms also take place within the services.
4. **Controllers** are used as the interaction point from the backend and frontend applications, this is where data is sent and received from requests and responses, and is used to administer the application's functionalities, sending, and retrieving data from the appropriate service to provide the required output. These are also used to display any issues which may have occurred in the Rest API.
5. **Exceptions** are thrown when something goes wrong and are caught in the controllers, to display the appropriate HTTP status code related to the exception being thrown.
6. **Converters** are used to convert objects from one type to another, such as converting from an Account to an AccountEntity to be able to interact with the database. Converters are static and are called from the service classes for any type conversions which need to take place.
7. **Validators** are used to validate an object, for instance to validate that all fields of an article are valid before creating it, this could include checks such as checking whether the article already exists in the database, and when suitable these classes throw exceptions.

The way in which my application is structured is aimed at maintaining SOLID, thus making the code maintainable as well as organized and easy to read and understand.

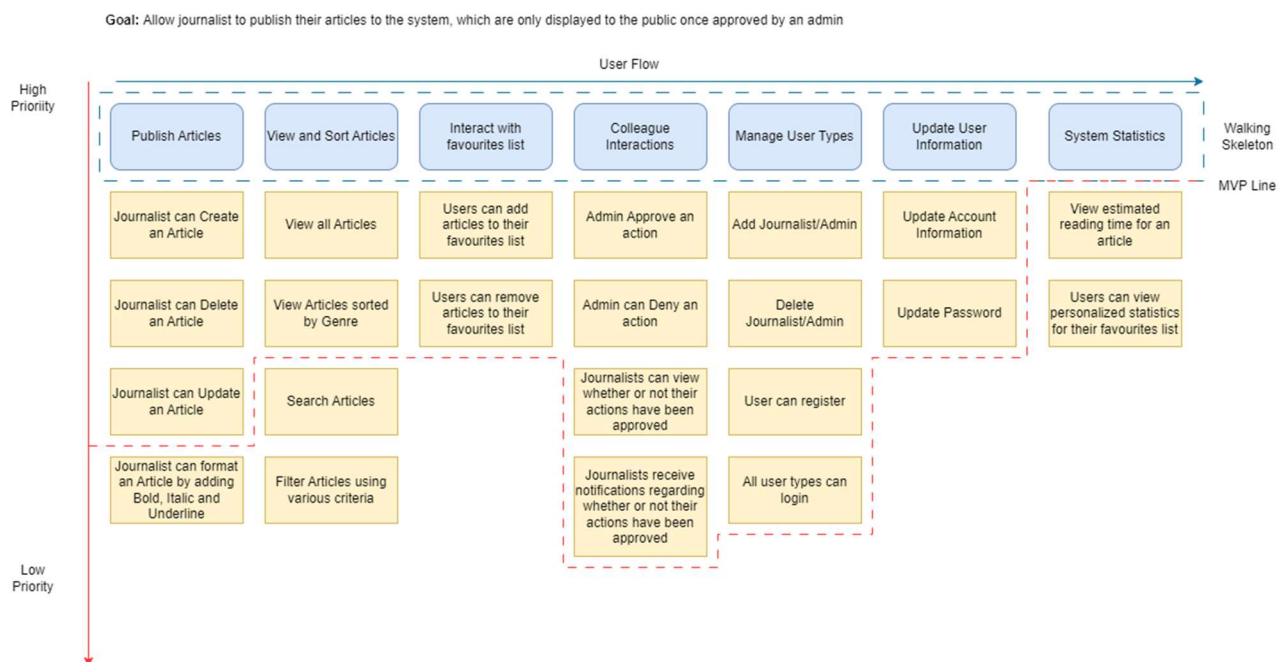
Introduction to C4

The C4 model is a set of hierarchical diagrams to demonstrate the architecture of the system. The role of a C4 model is acting as a centralized unit to system architecture for more efficient planning and communication in terms of creating the system architecture. A C4 model is used by system architects with developers to document system architecture choice. The C4 model is a centralized form of documentation for system architecture, in the sense that it is a well-known standard for making architectural documentation among developers.

There are 4 components to a C4 model, consisting of the following: C1 (System Context), C2 (Containers and Technology Choices), C3 (Components and their interactions) and C4 (Implementation Details).

Walking Skeleton

The walking skeleton displays the functionalities of the web application, some of these are split referring to certain user types, which are: User (End-User), Journalist and Admin. When a user type is not specified in a feature, for example under “View and Sort Articles” there is “View all articles”, this does not refer to a specific user type, it refers to all user types including website visitors that are not logged in.

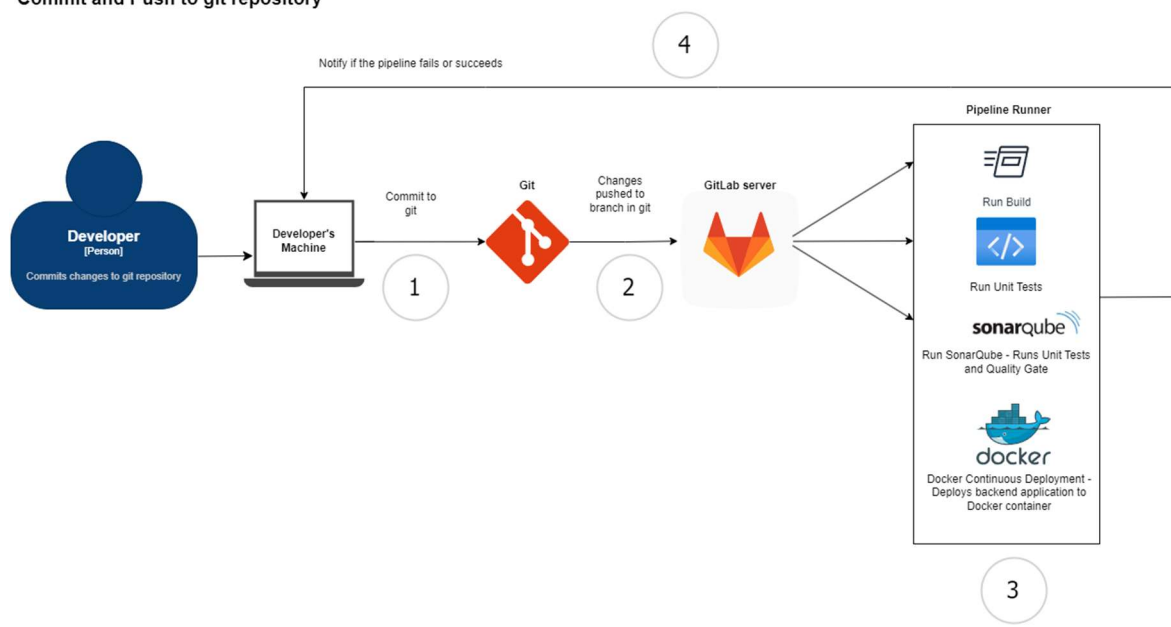


CI Setup

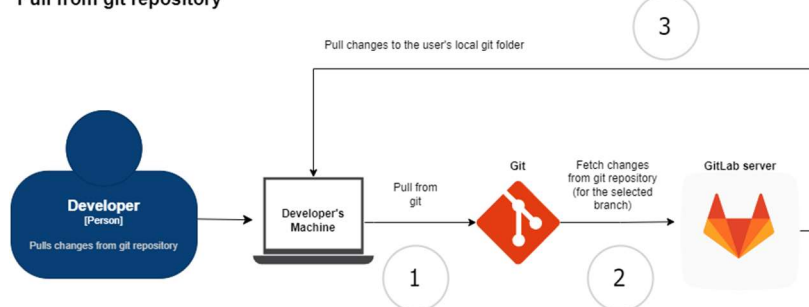
The CI setup diagram represents the way in which the CI (continuous Integration) pipeline is setup in the git repository. This shows the interactions between the developer's machine with the GitLab repository and the CI pipelines, in the event of the developer either committing and pushing to git or pulling the latest changes from the repository.

The CI consists of 4 pipelines: one responsible for the build, thus ensuring that the build is successful, one is responsible for the tests, therefore responsible for ensuring that all unit tests included in the project are passing, and the third pipeline is responsible for the SonarQube, so responsible to ensure that the unit tests and quality gate are passing in SonarQube, the final pipeline is responsible to the Continuous Deployment in Docker, this creates an image for the backend application, and then loads this in the backend container and starts it, also the database docker container is started, where both containers are run under the same bridge network.

Commit and Push to git repository

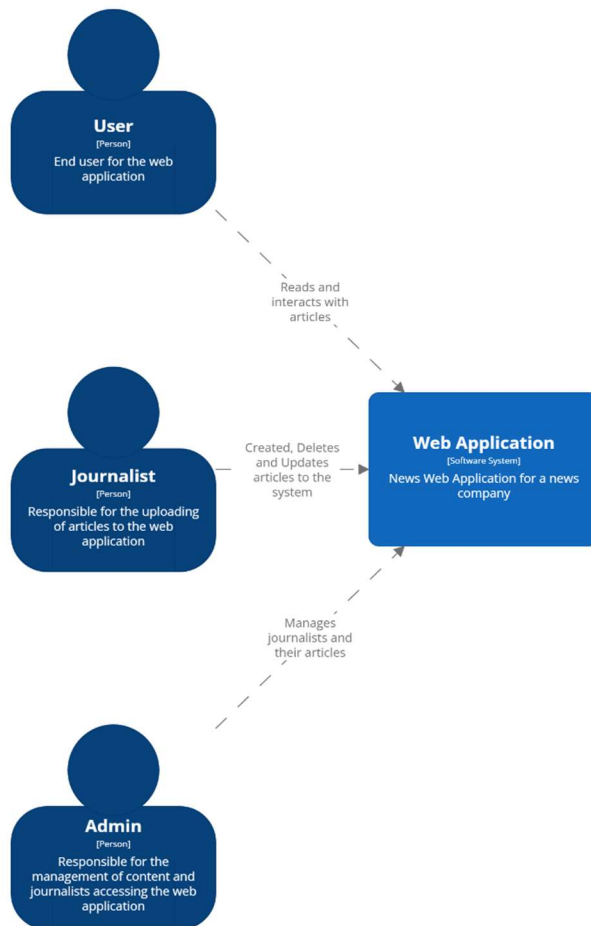


Pull from git repository



C1 (System Context)

C1 represents the system context. It consists of the system, users, and dependencies. My C1 consists of my 3 user types: User, Journalist and Admin, and their relationship and use of the News Web Application.



[System Context] Web Application

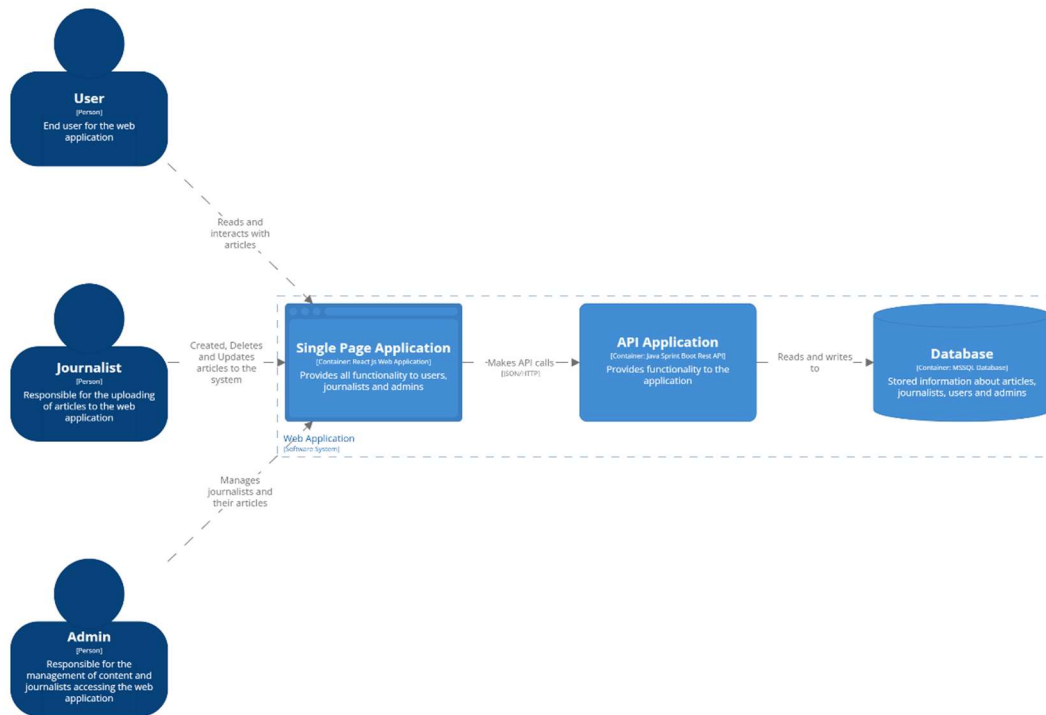
Thursday, 5 October 2023 at 11:01 Central European Summer Time

Key:



C2 (Containers and Technology Choices)

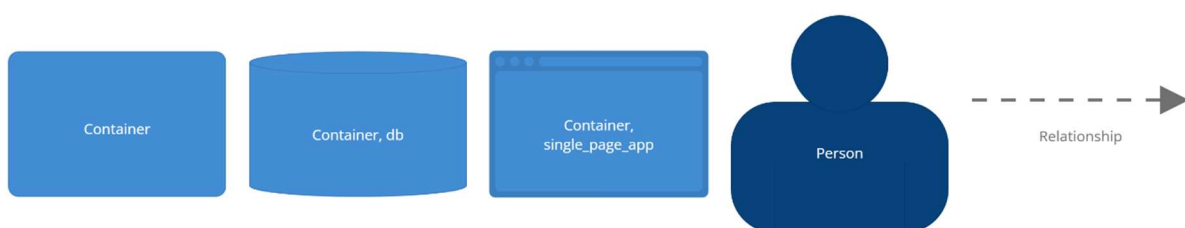
C2 represents the major containers and choices in my project, as well as the user types. My application consists of a Single Page Application created using React JS, as well as an API Application, which is a Java Sprint Boot Rest API, and an MSSQL relational database.



[Container] Web Application

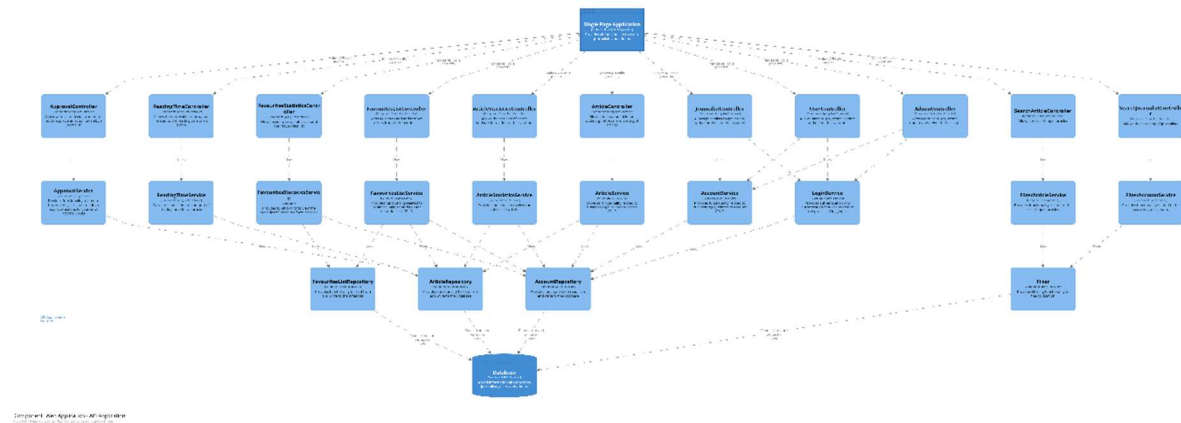
Sunday, 12 November 2023 at 17:33 Central European Standard Time

Key:



C3 (Components and their Interactions)

C3 represents the system components in greater depth, consisting of individual classes along with the major containers of the project (such as the single-page application). My C3 diagram represents the architecture of my application consisting of all controllers, services, and repositories, as well as their relationships with each other and with the single-page application and database.



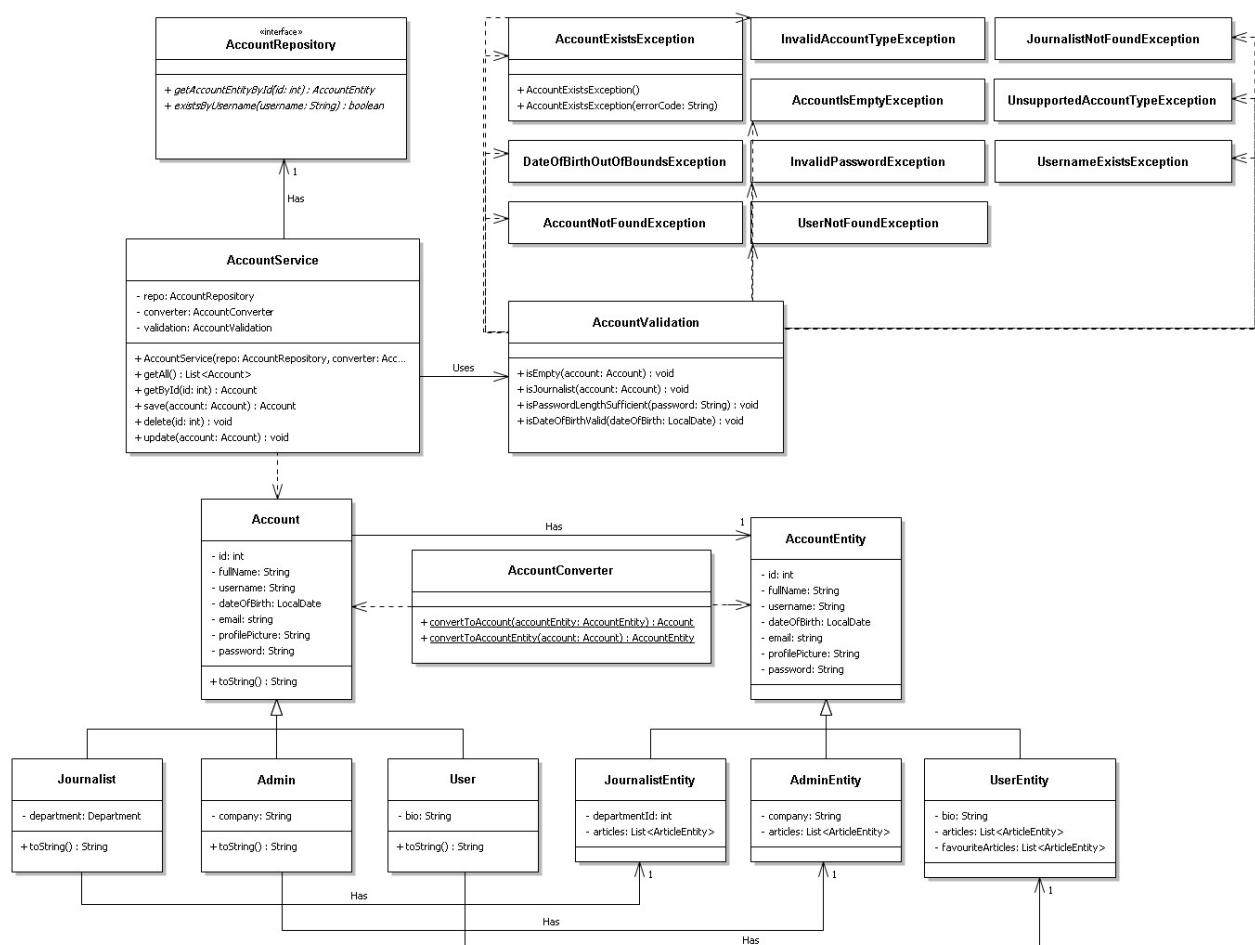
Key:



C4 (Code Diagram)

C4 represents the UML Class Diagram (UML Class Diagram). The UML Class Diagram included below consists of two important sections of my Rest API, which are the part in which inheritance is applied, where Journalist, User and Admin inherit from the Account, and where the Strategy Pattern is implemented to filter objects by searches in the app. The below account consists of all following components and their relationships with each other: repository, service, domain classes and subclasses, exceptions, validation classes, converter classes and entities (used with JPA).

1. Section Diagram for Inheritance (Highlighting an important section of the UML Class Diagram)



SOLID principles are applied to ensure a good code quality in the code of my application in terms of Object-Oriented Development.

Single Responsibility Principle is applied to my classes to ensure that each has only one responsibility. Below are some examples of how I implemented this principle in my app, in terms that each class has its own, individual responsibility:

1. Use of an ArticleRequest when creating an article, this divides the responsibilities between the article class and the request, in which the request has fewer fields, and thus only provides the necessary fields for creating or updating an article.
2. Use of a LoginService, this separates the responsibility of logging in a user, separating it from other CRUD related to the user.
3. Use of separated validation classes, such as AccountValidation for validating the user, and ArticleValidation for validating article fields.

The **Liskov Substitution Principle** is applied between the Account class and the User, Journalist and Admin classes where inheritance is applied, therefore any of the subclasses can be used in exchange of a base class, maintaining the expected behaviour of the application. The application of the Liskov substitution principle ensures a subclass can be used to replace a base class, for instance when creating an Account, the AccountService takes an argument of type Account in order to achieve this, given that inheritance has been applied, depending on the use case, either an Admin, Journalist or User can be passed and used to create that specific type of user, hence replacing the base class.

For the **Interface Segregation** principle, I am using interfaces, each with their own, individual responsibility (obviously with exception of the Filter interface used to implement the strategy pattern). The interfaces I use are designed to group together CRUD functionality related to its specific object. An example of how I implemented this in the AccountRepository interface is by only placing the methods directly related to the Accounts here, this therefore consists of just 3 methods which perform the following: get account by id or username and check if an account exists by username.

For the **Dependency Inversion** principle, I am using Dependency Injection (which is part of this principle), which is used within the service classes of my application to inject all necessary dependencies within the constructor of the class to make use of that repository (each repository has a separate interface). Some examples of the dependencies injected into the Service class are the following:

1. Repositories
2. Validation classes
3. Password Encoder/Token Encoder and Decoder (Only relevant for LoginService and AccountService)
4. Unit Tests.